

리눅스 네트워크 스택으로 읽는 Pod 네트워킹

1. Pod 네트워크는 누가 만드는가

1.1 컴포넌트별 책임

무엇을	누가	참고
netns 생성	CRI 런타임 (containerd / CRI-O)	kubelet이 아님
Pod에 IP 부여	CNI plugin	kube-proxy는 관여 X
Pod ↔ Pod 도달성	CNI plugin	kube-proxy 없이도 동작
ClusterIP → Pod 전달	kube-proxy (또는 CNI의 kube-proxy replacement)	CNI가 담당 X
노드별 Pod CIDR 배정	controller-manager 또는 CNI 자체 IPAM	구현마다 다름
위 전부의 실제 수행	리눅스 커널	

위 컴포넌트들이 하는 일은 결국 커널에 netns를 만들고, veth를 연결하고, 라우팅 테이블과 netfilter 룰을 채우는 것

CNI와 kube-proxy는 서로 다른 문제를 담당

- CNI는 Pod에 IP를 주고 서로 닿게 하는 것을 담당
- kube-proxy는 실재하지 않는 가상 IP(Service)를 실재하는 Pod로 보내는 것을 담당

1.2 CNI 계층의 필요성

쿠버네티스 네트워크 모델은 Pod 간 NAT을 금지한다. 그 결과 “Pod IP를 클러스터 전역에서 라우팅 가능하게 만들라”는 요구가 생기고 이 요구를 구현하는 것이 CNI 플러그인의 역할

Flannel, Calico, Cilium의 차이는 이 요구를 구현하는 방식의 차이

2. Network Namespace, veth, bridge

2.1 netns가 격리하는 것

netns 하나는 독립된 네트워크 스택 한 벌

격리 대상	확인 명령	의미
네트워크 인터페이스	<code>ip link</code>	Pod마다 자기 <code>eth0</code>
라우팅 테이블	<code>ip route</code>	Pod마다 다른 default gw 가능
ARP / neighbor 테이블	<code>ip neigh</code>	Pod 간 ARP 캐시 독립
netfilter 룰	<code>iptables -S</code>	Pod 안에서 보는 룰과 호스트의 룰은 별개
conntrack 테이블	<code>conntrack -L</code>	Pod별 연결 추적
소켓 / 포트 번호 공간	<code>ss -tlnp</code>	Pod마다 80 포트 사용 가능
<code>net.*</code> sysctl	<code>sysctl</code> <code>net.ipv4.ip_forward</code>	<code>securityContext.sysctls</code> 로 Pod별 조정

커널은 하나뿐이므로 커널 버전, 로드된 모듈, `nf_conntrack_buckets` (conntrack 해시 테이블 크기) 같은 모듈 단위 파라미터는 모든 Pod가 공유. netns가 나누는 것은 **자료구조이지 커널 코드가 아니다**

```
ip netns list           # 이름있는 netns만. 컨테이너 netns는 보이지 않는다
lsns -t net            # 시스템의 모든 netns + 소속 프로세스
nsenter -t <PID> -n ip addr
```

현상	근거
같은 Pod 컨테이너끼리 <code>localhost</code> 통신	netns 공유
같은 Pod 안에서 포트 중복 불가	netns 공유 (동일 포트 공간)
앱이 재시작되어도 Pod IP 유지	pause가 netns를 계속 점유

NetworkPolicy가 netns 격리 덕분에 가능해 보이지만, 실제 집행은 대부분 **호스트 netns 쪽 veth**에서 이루어진다. Pod 안에서 `iptables -S` 를 쳐도 정책 룰이 보이지 않는 이유다.

pause 컨테이너의 존재 목적은 netns 유지 그 자체다. 앱 컨테이너가 CrashLoopBackOff로 반복 재시작되어도 Pod IP가 변경되지 않는 이유이다.

2.2 veth pair

- 항상 쌍으로 생성되는 가상 이더넷
- 한쪽에 넣은 프레임이 반대쪽에서 그대로 나온다.
- 커널 내부에서 완결되므로 유저스페이스 왕복이 없다. (tap 디바이스와의 차이)

- 표준 패턴: 한쪽 끝은 Pod netns의 `eth0`, 다른 쪽은 호스트 netns의 `vethXXXX` / `caliXXXX` / `lxcXXXX`

```
# Peer 추적
# Pod 안에서: eth0@if10 → 호스트의 ifindex 10 이 peer
ip -d link show eth0
# 호스트에서: veth-pod1@if9 → 그 netns 의 ifindex 9 가 peer
ip -d link show veth-pod1          # 출력에 link-netns pod1 표시
cat /sys/class/net/<if>/iflink
```

2.3 CNI 구현 분기점

veth의 호스트 쪽 끝을 어디에 연결하는가가 CNI 구현 차이의 대부분을 결정한다.

CNI	호스트 쪽 끝	결과
Flannel / kindnet / bridge plugin	<code>cni0</code> 브리지에 연결	같은 노드 Pod는 L2 스위칭
Calico	연결하지 않음. IP도 없음. <code>/32</code> scope link 라우트만	같은 노드 Pod도 L3 라우팅
Cilium	<code>tc</code> / <code>tcx</code> 후에 eBPF 프로그램 부착	커널 netstack 상당 부분 우회
AWS VPC CNI	보조 ENI에 연결	Pod IP가 VPC 네이티브

2.4 bridge

소프트웨어 L2 스위치다. **MAC 학습(FDB)** 후 포워딩하며, `cni0` · `cbr0` · `docker0` 가 모두 이것이다. 브리지에 IP를 부여하면 호스트의 L3 인터페이스도 겸하게 되어 Pod의 default gateway 역할을 한다.

```
bridge link show          # 브리지에 붙은 포트
bridge fdb show br cni0   # MAC 학습 테이블
ip -d link show cni0      # STP, aging 등 파라미터
```

구조적 제약 1 — 브리지 트래픽은 netfilter를 통과하지 않는다

브리지를 지나가는(같은 노드 Pod ↔ Pod) 트래픽은 L2 포워딩이므로 L3 netfilter 혹은 거치지 않는다. 이를 강제로 통과시키는 것이 `br_netfilter` 다.

```
modprobe br_netfilter
sysctl -w net.bridge.bridge-nf-call-iptables=1
sysctl -w net.bridge.bridge-nf-call-ip6tables=1
```

kubeadm이 이 sysctl을 preflight로 검사하는 이유가 여기에 있다. 커널 5.3부터 netns별 sysctl이 되었으며, 그 이전에는 전역이었다. 부작용으로 브리지 트래픽에 conntrack 엔트리가 생성된다.

구조적 제약 2 — 브리지 자체의 비용

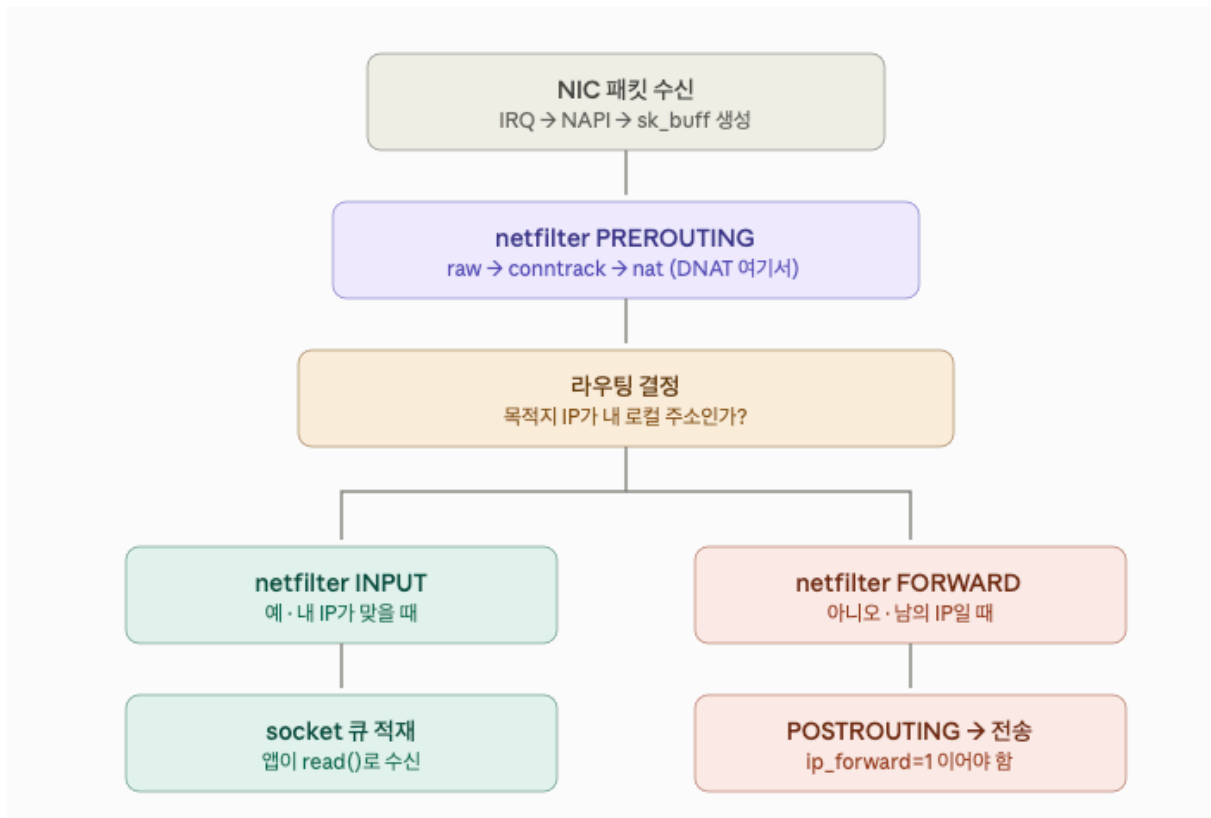
비용	내용
L2 브로드캐스트 도메인	노드의 Pod N개가 하나의 도메인. ARP/GARP/IPv6 NS·RA가 N개 veth에 플러딩
FDB 상태	MAC 학습 테이블 유지(기본 aging 300초), 프레임마다 학습 로직
서브넷 구조 고정	노드당 하나의 연속 서브넷에 묶임 → IP 효율 저하

3. TCP/IP Stack

3.1 계층과 매핑

계층	단위	쿠버네티스에서 보이는 것
L2 Link	Frame / MAC	veth, bridge, macvlan, ARP, VXLAN 내부 프레임
L3 Network	Packet / IP	Pod IP, Pod CIDR, 라우팅, overlay(VXLAN/IPIP)
L4 Transport	Segment / Port	Service port, targetPort, conntrack
L7 Application	HTTP/gRPC	Ingress, Gateway API, Service Mesh

3.2 수신 경로



라우팅 결정은 “이 패킷의 목적지 IP가 내가 가진 주소인가?”를 판단한다. 커널은 `local` 라우팅 테이블(내 인터페이스에 바인딩된 주소 목록)을 보고, 있으면 위로(INPUT), 없으면 옆으로(FORWARD) 보낸다.

노드에 `ip_forward=1` 이 필요한 이유

Pod가 외부로 보내는 패킷의 목적지는 노드 자신이 아니다. 목적지가 `8.8.8.8` 이면 노드 입장에서 “내 주소 아님”이므로 FORWARD 경로를 탄다.

문제는 리눅스가 기본적으로 라우터가 아니라는 점이다. `net.ipv4.ip_forward=0` 이면 커널은 남의 패킷을 대신 전달하는 일을 거부하고 그 자리에서 드랍한다. 로그가 남지 않으므로 증상은 “Pod에서 외부가 안 됨”으로만 보인다. kubelet과 CNI 플러그인이 시작 시 이 값을 1로 세팅하는 이유다.

노드 자신이 주고받는 트래픽(SSH, kubelet API 등)은 INPUT/OUTPUT 경로이므로 `ip_forward` 와 무관하다. **지나가는 트래픽만 영향을 받는다.**

DNAT이 PREROUTING에 있는 이유

순서가 전부다. 라우팅 결정은 그 시점의 목적지 IP를 보고 판단하는데, DNAT은 목적지 IP를 바꾸는 작업이다. 따라서 반드시 라우팅 결정보다 앞에 있어야 한다.

DNAT을 라우팅 뒤에서 하면, 커널은 아직 바뀌지 않은 ClusterIP를 기준으로 출구 인터페이스를 이미 결정해버린 상태가 된다. 그 뒤에 목적지를 Pod IP로 바꿔도 출구는 엉뚱한 곳으로 고정되어 있다. PREROUTING에서 먼저 진짜 목적지로 바꿔놓아야 라우팅 결정이 그에 맞는 경로를 고른다.

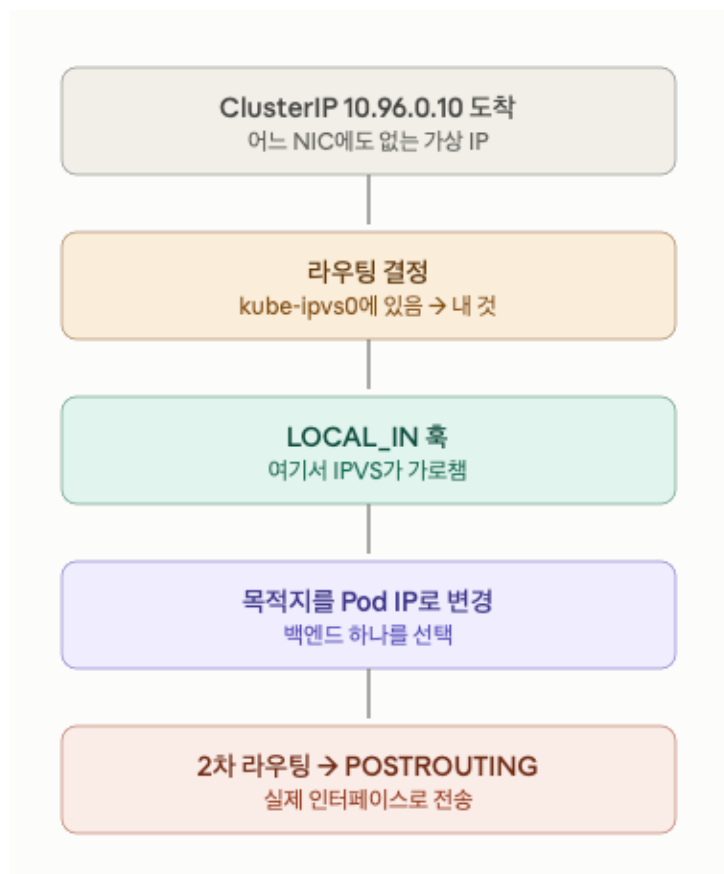
같은 이유로 **SNAT은 반대편인 POSTROUTING에 있다**. 출발지 IP는 라우팅 결과(어느 인터페이스로 나가는가)를 알아야 정할 수 있기 때문

IPVS 모드가 더미 인터페이스를 필요로 하는 이유

iptables 모드는 PREROUTING에서 DNAT을 하지만, IPVS는 **LOCAL_IN** 훅에서 동작한다. **LOCAL_IN**은 위 그림의 왼쪽 가지, 즉 "내 것으로 판정된 패킷"만 지나가는 자리

그런데 ClusterIP(**10.96.0.10** 등)는 어느 노드의 어느 NIC에도 붙어있지 않은 가상 주소다. 그대로 두면 커널은 "내 주소 아님"으로 판단해 FORWARD로 보내버리고, IPVS는 패킷을 보지도 못한다.

그래서 kube-proxy는 **kube-ipvs0** 더미(dummy) 인터페이스를 만들고 모든 ClusterIP를 여기에 바인딩한다. 더미 인터페이스는 실제로 패킷을 주고받지 않고 **"이 주소는 내 것"이라고 커널에 등록하는 용도로만 존재한다**. 이러면 라우팅 결정이 YES를 내놓고, 패킷이 **LOCAL_IN**까지 올라와 IPVS의 손에 들어간다.



첫 번째 라우팅은 “이것이 내 것인가”를 판단해 패킷을 위로 올려보냈고, IPVS가 목적지를 진짜 Pod IP로 바꾼 뒤에는 “이 Pod로 가려면 어디로 나가야 하는가”를 다시 물어야 한다. 한 패킷이 라우팅 테이블을 두 번 조회하는 셈 (2차 라우팅이라고 하는 이유)

3.3 송신 경로



write() 는 “보냈다”가 아니라 “맡겼다”

앱이 `write()` 를 호출하면 데이터가 유저 공간에서 커널의 소켓 송신 버퍼로 복사되고 함수는 바로 리턴한다. 이 시점에 패킷은 아직 네트워크에 나가지 않았다.

따라서 버퍼가 가득 찼을 때의 동작이 중요해진다. 블로킹 소켓이면 `write()` 가 그 자리에서 멈추고, 논블로킹이면 `EAGAIN` 을 반환한다. 애플리케이션이 느끼는 백프레셔의 정체다. 버퍼

크기는 `net.ipv4.tcp_wmem` 으로 조절한다.

TCP: 자르고 번호 붙이고 사본을 남긴다

데이터가 MSS 단위로 잘려 세그먼트가 되고 각각에 시퀀스 번호가 붙는다. 동시에 혼잡 윈도우와 상대의 수신 윈도우를 보고 지금 몇 바이트까지 내보낼 수 있는지 계산한다.

보낸 뒤에도 **재전송 큐에 사본을 남긴다**. ACK가 와야 그 사본을 버리므로, 소켓 버퍼가 비었다고 전송이 끝난 것이 아니다.

최근 커널은 TSO/GSO 때문에 실제로는 여기서 자르지 않고 큰 덩어리를 그대로 내려보낸 뒤 NIC이 하드웨어에서 자르게 한다. `tcpdump` 로 잡으면 MTU보다 큰 패킷이 보이는 이유

라우팅이 netfilter보다 먼저다

수신 경로와 순서가 뒤집혀 있다. 수신은 `PREROUTING` → 라우팅 이지만 송신은 `라우팅` → `OUTPUT` 이다.

로컬에서 만들어진 패킷은 출발지 IP조차 정해지지 않은 상태이기 때문이다. 어느 인터페이스로 나갈지를 먼저 알아야 그 인터페이스의 주소를 출발지로 채울 수 있고, 그래야 netfilter 룰이 온전한 패킷을 보고 판단할 수 있다.

여기서 함정이 하나 생긴다. `OUTPUT` 체인에서 DNAT을 걸면 라우팅이 이미 끝난 뒤에 목적지가 바뀌는 셈이라 커널이 라우팅을 다시 돌려야 한다(reroute). 노드 자신이 ClusterIP로 접속할 때 kube-proxy가 `KUBE-SERVICES` 룰을 `PREROUTING` 뿐 아니라 `OUTPUT` 에도 거는 이유이기도 하다.

`POSTROUTING` 은 마지막 관문이므로 SNAT과 masquerade가 여기 붙는다. Pod IP를 노드 IP로 바꿔 외부로 내보내는 작업이 이 지점에서 일어난다.

neighbor: IP만으로는 못 보낸다

목적지 IP와 출구 인터페이스는 정해졌지만, 실제로 선에 실으려면 next hop의 MAC 주소가 필요하다. 커널은 neighbor 테이블(`ip neigh` 로 확인하는 ARP 캐시)을 조회하고, 없으면 ARP 요청을 보낸 뒤 응답이 올 때까지 패킷을 붙잡아 둔다.

캐시가 꽉 차거나 대기 큐가 넘치면 패킷은 조용히 버려진다. Pod 밀도가 높은 노드에서 `gc_thresh` 튜닝이 필요해지는 지점

qdisc: tc가 동작하는 곳

이더넷 헤더까지 붙은 완성된 패킷이 인터페이스별 큐에 들어간다. 커널은 여기서 언제 얼마나 내보낼지를 결정한다. 기본값은 배포판에 따라 `pfifo_fast` 또는 `fq_codel`

CNI의 `bandwidth` 플러그인은 Pod에 다음 애노테이션이 붙어 있을 때 개입한다.

```
kubernetes.io/egress-bandwidth: 10M
kubernetes.io/ingress-bandwidth: 5M
```

기준은 **Pod 입장**이고, 플러그인이 손대는 것은 **호스트 쪽 veth**다. 이 둘의 방향이 뒤집혀 있어 구현이 비대칭이 된다.

애노테이션	Pod 기준 방향	호스트 veth 기준	구현
<code>ingress-bandwidth</code>	Pod로 들어옴	veth의 송신	veth에 <code>tbft</code> 하나
<code>egress-bandwidth</code>	Pod에서 나감	veth의 수신	<code>ifb</code> + <code>mirred</code> 리다이렉트 후 <code>tbft</code>

qdisc는 송신 방향에만 존재하므로, 수신 방향을 제한하려면 셰이핑할 큐를 새로 만들어야 한다. `ifb` (Intermediate Functional Block)가 그 역할이다. 수신 트래픽을 `ifb`로 넘긴 뒤 그쪽 송신 큐에서 셰이핑한다. **결국 모든 셰이핑은 송신 경로의 qdisc를 빌려 쓴다**

드라이버와 NIC

qdisc에서 빠져나온 패킷은 드라이버가 링 버퍼(TX ring)에 채워 넣고 NIC에 알린다. NIC이 DMA로 메모리에서 직접 읽어 송출하고, 완료되면 인터럽트를 올려 커널이 `sk_buff`를 해제한다.

3.4 MTU

- 이더넷 기본 MTU 1500
- 오버레이는 헤더를 덧씌우므로 **payload가 줄어든다**

방식	오버헤드	권장 Pod MTU
VXLAN (IPv4)	50 B	1450
VXLAN (IPv6)	70 B	1430
IP-in-IP	20 B	1480
WireGuard (IPv4)	60 B	1440
GENEVE	가변 ≥50 B	≤1450
없음 (BGP / host-gw)	0	1500

4. Routing

4.1 라우팅 결정 순서

리눅스는 policy routing을 사용한다.

- ① ip rule 을 우선순위(낮은 수부터) 순회
매칭된 rule 이 가리키는 테이블을 조회
→ 그 테이블에 답이 없으면 다음 rule 로 계속 (테이블 하나로 끝나지 않는다)
- ② 선택된 테이블에서 Longest Prefix Match
/32 > /26 > /24 > /16 > 0.0.0.0/0
- ③ prefix 길이가 같으면 metric 이 낮은 것
- ④ 그래도 동률이면 ECMP (멀티패스 해싱)

```
ip rule show # 기본: local 0, main 3276
6, default 32767
ip route show table main
ip route show table all # Calico/Cilium이 별도 테이블
을 쓸 수 있음
ip route show table local # 여기 있으면 "내 것"으로 판정
된다
ip route get 10.244.1.5 # 커널
의 실제 결정
ip route get 10.244.1.5 from 10.244.0.11 iif cali-a # 조건
부 시뮬레이션
```

`ip route` 가 테이블의 내용을 보여준다면, `ip route get` 은 커널이 실제로 내리는 결정을 보여줌

5. Q&A

Q1. CNI가 IP를 주고 라우팅까지 설정한다면 kube-proxy는 왜 따로 필요한가

CNI가 다루는 주소는 전부 **실재하는 주소**다. Pod IP는 어느 노드의 어느 veth 뒤에 실제로 존재하므로 라우팅 테이블에 넣을 수 있다. 반면 ClusterIP는 실재하지 않는다. `10.96.0.10` 을 소유한 인터페이스는 클러스터 어디에도 없다. 라우팅은 "이 주소가 어디 있는가"를 푸는 도구이므로, 애초에 어디에도 없는 주소는 라우팅으로 풀 수 없다. 필요한 것은 **주소를 바꾸는 일(DNAT)**이고 그 자리가 3.2에서 본 netfilter 혹이다.

	푸는 문제	도구
CNI	실재하는 주소 사이의 도달성	라우팅 테이블, veth, 오버레이
kube-proxy	가상 주소를 실주소로 번역	netfilter DNAT, IPVS

그래서 kube-proxy가 죽어도 Pod ↔ Pod 통신은 멀쩡하고 Service만 안 된다. 거꾸로 CNI가 망가지면 둘 다 안 된다. 장애를 만났을 때 이 구분이 원인을 반으로 줄여준다.

Cilium의 kube-proxy replacement는 이 번역을 iptables/IPVS 대신 tc eBPF 혹에서 수행한다. 역할의 경계가 사라진 것이 아니라 구현 위치가 옮겨간 것

Q2. 같은 노드의 Pod끼리 통신할 때도 Service 규칙과 NetworkPolicy가 적용되는가

CNI 구현에 따라 다르다.

CNI	같은 노드 Pod 간 경로	netfilter 통과 여부
Flannel / kindnet / bridge	<code>cni0</code> 안에서 L2 스위칭 으로 종료	기본적으로 통과하지 않음
Calico	호스트 veth에 IP 없이 <code>/32</code> 라우트로 L3 포워딩	항상 통과
Cilium	tc/tcx eBPF 혹에서 처리	netfilter 경로에 의존하지 않음

브리지 계열이 문제가 되는 이유는 단순하다. L2 포워딩은 L3 혹에 올라오지 않으므로 iptables 룰이 적용될 **기회 자체가 없다**. 이를 강제로 올려주는 것이 `br_netfilter` 이고, kubeadm이 `bridge-nf-call-iptables` 를 preflight로 검사하는 이유가 여기에 있다.

증상은 특유의 비대칭으로 나타난다. **다른 노드의 Pod와는 정책이 먹는데 같은 노드 Pod끼리만 안 먹는다**. 노드 간 트래픽은 어차피 L3 라우팅을 거치므로 혹을 통과하기 때문이다. 정책이 “가끔 안 듣는다”고 느껴지면 출발지와 목적지가 같은 노드인지부터 확인하는 편이 빠르다.

+) 용어

용어	뜻
netns	리눅스 커널 기능으로, 프로세스마다 독립된 네트워크 환경(IP, 라우팅 테이블, 인터페이스 등)을 제공. 컨테이너가 서로 격리된 네트워크를 가질 수 있게 해주는 기반 기술
veth	virtual ethernet pair. 커널 내부에서 완결되는 가상 케이블

용어	뜻
TAP 디바이스	리눅스 커널이 제공하는 가상 네트워크 인터페이스로, 이더넷 프레임(L2) 단위 로 동작하는 가상 NIC
FDB	Forwarding DataBase. 브리지의 MAC 학습 테이블
FIB	Forwarding Information Base. 커널 라우팅 테이블의 실체
pause 컨테이너	Pod의 netns를 소유하는 컨테이너. 앱 컨테이너 재시작 시에도 Pod IP를 유지시킨다
CRI	쿠버네티스가 컨테이너 런타임(containerd, CRI-O 등)과 통신하기 위한 표준 인터페이스. 쿠버네티스가 특정 런타임에 종속되지 않게 추상화 계층 역할
CNI	컨테이너에 네트워크를 붙여주는 표준 인터페이스/플러그인 규격. 파드가 생성될 때 netns를 만들고 IP 할당, 라우팅 설정 등을 담당 (ex. Calico, Flannel, Cilium)
proxy ARP	자신에게 할당되지 않은 IP에 대해 대리로 ARP 응답
on-link	해당 목적지가 이 디바이스에서 직접 도달 가능하다는 라우트 속성